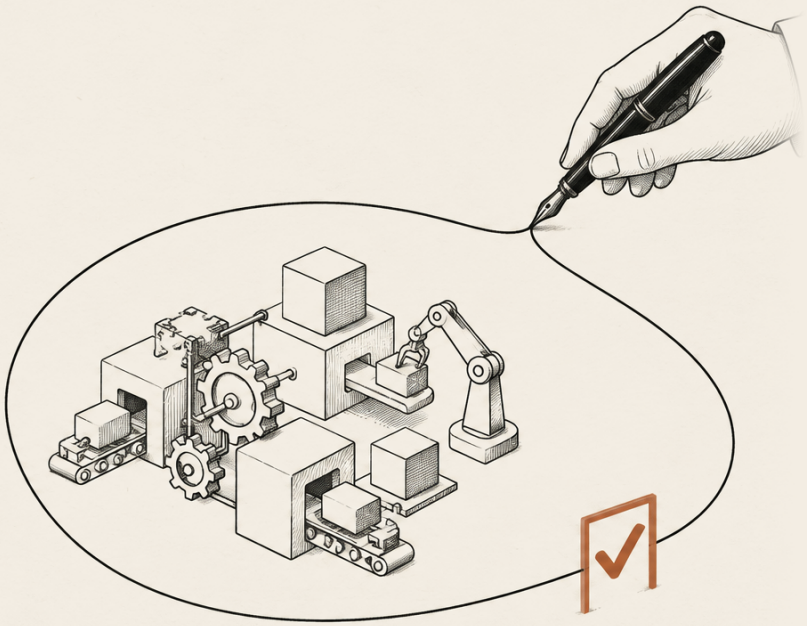
Vibe Coding 实战

用提示词组织工作
用证据判断结果

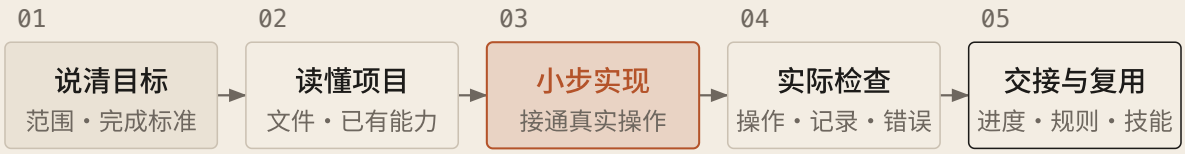


朱炳杰

2026 年 10 月

从想法到可以使用的结果

用自然语言说明目标，让编程智能体读取项目、修改文件、运行检查，再根据实际结果继续完善。智能体会反复调用工具，根据结果决定下一步，朝目标推进。



独立工作可并行；发现问题后回到对应步骤修正

一条主线，按需要增加方法

先说清目标，调研已有方案，再让 AI 实现、检查和修正。把需求、依据与进度留在项目文件中，按需要增加协作与自动化。重点是理解每种方法解决什么问题、何时使用；具体工具操作可以让 AI 查证后协助完成。

当前情况	对应做法
想法还比较模糊	澄清目标、范围和完成标准，先形成最小可用版本的计划
开始开发或修改	读取项目与相关资料，复用已有做法，小范围实现并及时运行
结果不符合预期	按真实操作验收，保留出错记录，先定位原因再修复
有多项独立工作	可同时做只读调查；统一安排改动，重要成果再独立审查
任务中断或重复出现	保存进度；把有效方法写成规则或技能，再考虑批量执行

每章浅色框里的提示词怎么用

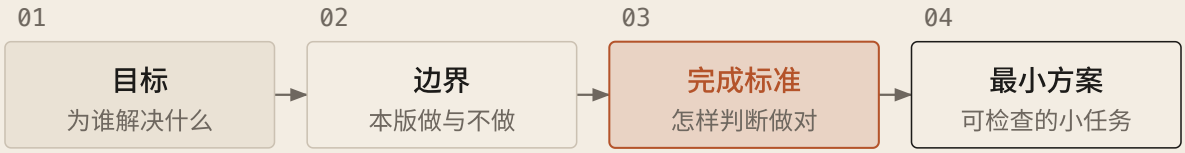
1. **找到并复制**。每章正文中的浅色大框就是提示词。按任务选一框，复制标题下的全部文字。
2. **替换方括号**。把“[目标]”等内容换成自己的要求，例如“添加待办事项，刷新后仍能看到”。
3. **粘贴并发送**。在 Codex、Claude Code 中打开项目，把改好的整段文字发到对话框，再按步骤运行并检查结果。

人的判断放在哪里

人决定目标、取舍和是否接受结果；AI 比较方案、执行并提供证据。这里的判断力（taste）指理解用户并确定优先级。演示版可侧重表达，日常工具可侧重可靠性；先满足必要要求，再权衡体验、成本与速度。用场景、样例与可观察标准表达偏好，并检验标准是否合适。

第 1 章

把想法变成可执行的任务



先明确重要选择，再安排实现

先把结果说清楚

1. **说明目标与依据**。写清谁要使用、要完成什么操作，附上已有资料或认可的参考。描述希望达到的结果，不必预先规定所有技术步骤。
2. **划定第一版**。区分现在必须完成与以后再做的内容。想法模糊时，让 AI 每次问一个关键问题，逐步明确场景、边界和取舍；已明确的要求无需反复确认。
3. **约定完成标准**。把“好用、完整、专业”改成可以观察的行为：哪些操作应成功、失败时怎样处理、哪些内容需要保留。先检查这一版需求是否遗漏，再安排实现。

需求整理与最小方案

我要让 [使用者] 在 [场景] 完成 [目标]。项目和参考资料在 [位置；尚无则注明]。本版必须完成 [范围]，暂不做 [内容]；允许修改 [范围]。
先核对现状，保留原始要求，在现有需求文档或当前功能的 spec.md 中写清目标、关键操作、完成标准、必须保留的行为和未知项。能查证的自行查证；关键需求不清时逐个询问并给建议，标明未确认的假设。
提出最小方案和有检查方法的小任务。本轮只明确需求与初步安排，不改代码；技术选择待调研。沿用已有文档或 Spec Kit 步骤；材料不可读先说明缺口，无法写文件则返回草稿，不声称已保存。

把重要选择留给人，把实现空间留给 AI

需求规格（Spec）写清目标、行为、约束和验收标准；计划写“准备怎么做”。可以先与 AI 讨论想法，再整理成规格；人重点审核目标、取舍和验收，技术方案经调研后再定。标准明确有利于自动执行，却不能保证理解和实现都正确，结果仍需核验。目标清楚的小改动可直接执行，不必另写长计划。

第 2 章

项目文件怎样分工和衔接



文件名为常见组织方式；Spec Kit 沿用当前选定的功能目录

文件把聊天中的要求留下来。图中以 / 结尾的是文件夹，`.md` 是可直接阅读、带简单排版的文字文件。`README.md` 说明启动和检查；`AGENTS.md` 是 Codex 等智能体支持的规则格式。Claude Code 是否直接加载它取决于版本与配置，也可在 `CLAUDE.md` 中用 `@AGENTS.md` 导入。`.specify/` 保存项目原则与流程支持文件，功能需求放在当前功能目录。

文件	负责回答的问题
`spec.md`	要做什么、哪些内容不做、看到什么结果才算完成；原始要求保留原文或可回查链接
`research.md`	哪些问题查过、依据在哪里、比较过什么方案、为什么选择、还有哪些未知；链接实验记录
`plan.md` / `tasks.md`	前者记录实现方法；后者列出任务顺序、状态和检查方法
`PROGRESS.md`	跨会话记录实际进展、未完成事项和下一次继续的位置
检查记录	实际运行了什么、针对哪个版本、结果和失败证据在哪里

整理项目文件与读取入口

项目：[路径]，任务：[目标]，允许更新：[文档范围]。先定位已有规则、需求、调研、方案、任务、进度与证据；只补缺失且有用的文件，沿用 Spec Kit 当前功能目录，不另建副本。路径不可读先报缺口；无写权限返回草稿。

规则只补项目约定、修改范围、检查命令和文档入口；调研与实验用简短索引指向原文。保留已有有效内容，不移动或删除原件。列出更新路径及维护条件；按工具要求刷新规则，核对加载来源，无法观察时标为未验证，不把创建文件当成已生效。

索引帮助查找，原文保留依据。会话压缩本身不会改写磁盘上的项目文件；关键信息应提前保存，并在后续会话中可访问。文件仍在不等于 AI 已读取，恢复时按索引重读当前规格、进度及相关证据。需求变更同步方案与任务，完成后更新结果。小任务可合成短记录，不必凑齐文件。

第 3 章

先读懂项目，再决定怎么改



简短目录指向当前文件，需要时再读取细节

先调研已有方案，再讨论架构

打开正确项目，先查已有实现和相似项目，再决定复用、改造或自建。依据官方文档、原始研究和示例，比较适用场景、成本及限制。讨论模块分工、数据流与接口（程序间交换数据的约定）；追问“为什么这样设计、何时不适用”，让新想法建立在已有工作之上。

开工前保护好数据

1. **用测试环境**。用数据副本和测试账号，限制读写与网络访问。扩大权限试验时隔离正式系统；删除、发布或修改正式数据前，明确授权、备份和恢复办法。
2. **保管密钥**。优先用密钥管理服务；本地 `.env` 仍需限制访问且不得提交，不贴进聊天或网页代码。`.gitignore` 不影响已跟踪文件，不阻止 AI 读取，也不清除已有历史；泄露后撤销并更换密钥。
3. **先查陌生内容**。网页或文件中的指令诱导 AI 偏离任务，叫提示注入。读取时避免同时开放私密数据与外发、删除权限；第三方技能、脚本和软件包先核对来源。

项目阅读与改动定位

目标：[当前任务]，材料：[位置]，允许更新的调研文件：[路径]。核对版本、已有改动、规则和运行条件；本轮不改程序，只在测试环境验证。材料不可读先报缺口，不能写记录则返回草稿。

查找已有实现与相似方案，比较复用、改造与自建；核对软件包、官方来源和使用许可。无法联网时标明外部资料未核实，不编造来源。围绕架构未知做最小验证，说明模块、数据流与依赖。

记录问题、来源与日期、观察、备选、选择理由及未知，区分事实、推断和假设；说明什么新证据会改变决定。报告实际改动与未验证项，保留原始资料和检查证据。

让调研形成决定

上下文是 AI 当前能看到的信息，应按需提供。调研要回答“能复用什么、为什么选它、什么还需验证”，并落实到方案。文件存在不等于已经读到；本地运行也不代表资料不会传给云端模型。

第 4 章

用 Spec Kit 串起文件与执行



新证据改变方案时，先更新对应文件，再调整受影响的任务

Spec 是需求规格；Spec Kit 是组织规格、方案与任务的可选开源工具。先理解方法，再按需选用；小改动用短记录即可。

步骤与对应文件：以安装版本为准

步骤	作用与需要检查的内容
`constitution`	建立项目原则，保存在 `.specify/memory/constitution.md`
`specify / clarify`	写清需求、澄清歧义，维护 `spec.md`
`plan`	调研写入 `research.md`，方案写入 `plan.md`
`checklist`	按需检查需求质量；通过不等于功能已实现
`tasks / analyze`	生成 `tasks.md`；按需核对需求、方案、任务是否一致
`implement`	按任务实施，运行检查，保留结果
`converge`	实施后找遗漏、追加剩余任务；仍需实际验收

`plan` 还生成 `data-model.md`（数据组织）、`quickstart.md`（验证步骤）；有对外接口时生成 `contracts/`（接口约定），供实现与验收使用。

当前默认技能入口：Codex 用 `$speckit-specify`；Claude Code、Copilot 用 `/speckit-specify`。当前功能由 `.specify/feature.json` 或运行设置指定，切换 Git 分支不会自动切换它。

按 Spec Kit 推进当前功能

项目：[路径]，依据：[要求与记录]，本轮步骤：[步骤名]，允许修改：[范围]。核对安装版本、调用入口、当前功能和前置文件；未安装或前置缺失先给补齐方案，不擅自安装或跳步。保留已有文件；澄清歧义，查证未知。需要自动测试时写入规格并拆成任务，每项对应需求与检查方法。

只有用户能调用时给出命令并等待执行结果。报告实际产物、检查、冲突及下一步；未调用不称已执行，要求矛盾不擅自取舍。

不熟悉命令时让 AI 查官方说明并核对入口。讨论后整理规格，人审核目标与验收，AI 按任务实施；需求变更同步方案与检查，避免新提示覆盖旧约定。文档齐全不等于完成。

第 5 章

小步实现，给出具体反馈



每轮聚焦一组相关改动，检查后再继续

围绕一段完整操作推进

1. **保留可恢复的版本**。Git 用来记录与比较版本。先保护已有改动，再用分支（独立的版本线）开展本轮工作。完成一段并检查通过后，提交一次本地版本，说明改了什么。提交是本地存档，推送是上传到远程仓库；工具的撤销记录不能替代版本保存。
2. **先接通一条真实路径**。让用户能从第一步操作走到真实结果，需保存的内容确实保存，再增加其他功能。界面工作可先看预览确认布局，再接真实数据；预览完成不等于功能完成。
3. **根据实际画面反馈**。提供截图或参考文件，说明“位置、当前表现、目标表现、保持不变的部分”。一次反馈聚焦一组相关问题，改后重新打开核对。

实现一轮可检查的改动

项目与需求：[位置；无文档则写明要求]。本轮完成 [功能范围]，按 [关键检查点] 推进；允许修改 [范围]，保留 [已有功能与样式]，参考 [文件或截图；无则注明]。
核对已有改动，用版本记录或备份保留可恢复基线，不夹带他人改动。按 [可运行的完成检查] 在 [预算] 内实现、检查和修复；缺少关键材料先说明，遇未授权操作、关键取舍或重复失败无新线索时暂停相关工作。接通入口、处理、保存与反馈；标明模拟数据、未完成功能或固定答案。
说明关键改动与数据去向，列出对应文件；运行项目，给出可打开的位置、检查步骤和实际结果。界面改动实际点击、输入、刷新并对照截图；缺工具则给人工步骤，保留未验证状态。单独保存本轮改动；仅在获准时本地提交，不推送。更新已有任务与进度；按 [位置、当前表现、目标表现、保持不变项] 继续修改，不重写无关部分，改变目标前说明取舍。

每一轮都应积累成果

修改前保留的可用版本叫基线。满意一版就保存其版本、运行方法和检查结果，再开始下一轮；对比新旧时使用同一标准。小步推进便于定位收益与退步，避免一次重写丢掉已认可的功能。先亲自走完关键操作，再给具体反馈；没有变好，就恢复基线或缩小改动。

第 6 章

从实际使用判断是否完成



运行没有报错，仍需核对是否完成原来的目标

检查用户真正会经历的情况

检查方向	需要看到的结果
正常操作	从实际入口完成目标，显示结果与实际保存内容相符
错误与重复	缺少必要输入、操作失败或重复执行时，有合理处理，不意外重复生效
重开与恢复	刷新、重启或中断后，应该保留的内容仍在，可以按约定继续
权限与原有功能	没有权限的账号不能读取或修改数据；相关已有行为仍正常

按需求运行验收

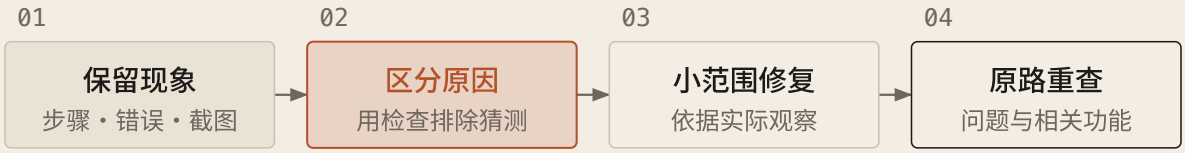
根据 [需求与原始材料] 检查 [项目位置与版本]，允许测试的环境与数据：[范围]。先列预期，重要歧义先确认，不抄程序输出作正确答案。本轮只验收，不改实现或放宽标准。
从真实入口走完整操作，核对显示与保存记录；按风险检查失败、重复、重开、恢复及旧功能。有测试需求时在获准范围补充检查；验证修复应能复现原错误，再在修复后通过。有账号或公开入口时，检查无权限访问、密钥暴露与外部输入被当命令执行的风险，禁止越权测试。
逐项报告操作、预期、观察、证据及通过、失败或未验证。注明模拟数据、替代服务和环境限制。检查有外部影响时遵守授权；缺工具或不能运行则列人工步骤与未验证项。不以没报错代替完成，不删、跳过或放宽检查换取通过；要求与检查冲突时说明。

把检查结果与完成承诺对应起来

端到端检查就是像用户一样从第一步做到最后，确认中间环节接得上。先确认检查能发现缺失或错误，再看修改后通过，才能知道它真能发现问题。收到报告时，对照要求查看操作证据；缺少证据仍是未验证。程序和 AI 自写的检查可能共享误解。功能正常也不代表安全；向真实用户开放前，另查权限、代码与记录中的密钥及恢复办法，重要问题请有经验的人复核。

第 7 章

出错时，先定位再修改



每次尝试都应增加证据，避免无依据地重写

把一次失败变成可用线索

1. **保留完整现象**。记录操作步骤、预期与实际结果、错误消息原文（去掉密码、密钥等敏感信息）、相关截图，以及问题发生前的改动。不要只留下“不能用”。
2. **先区分原因**。让 AI 核对文件位置、运行设置、权限和数据，再提出可能原因及能区分它们的检查。每次尝试都应增加一条证据。
3. **只修证据指向的范围**。修复后重走原操作，再检查相关原有功能。偶发故障记录尝试次数和出错次数；暂时不再出现，不等于原因已经解决。

定向排错

问题：[步骤、预期、实际]，项目与错误记录：[位置]，最近改动：[内容；未知则注明]。允许修复 [范围]，调查预算：[时间或次数]。
先核对版本、运行设置、权限和文件，暂不重写；在获准测试环境复现。不能稳定出现则记录条件、尝试和出错次数；材料或工具不足先报缺口，不猜成定论。
列出可能原因及预期观察，做最小检查，找最早偏离。记录关键实验的假设、版本、条件、操作、结果及证据；保留无效路径。证据支持后在允许范围小改；能自动复现则保留检查，核对修复前失败、修复后通过，再查相关功能。重复无效或预算将尽时保存事实与未知，必要时转交审查；无写权限返回记录，不含敏感信息。

失败记录也是项目资产

同样条件下再次出错叫复现；修改后旧功能坏了叫回归。保留失败样例、排除理由和修复后的检查，下次遇到相似问题先查记录，避免重复无效尝试。旧结论只在对应条件下成立，版本或环境变化后仍可能需要重试。把已修复问题加入回归检查，比只写“以后注意”更可靠。

第 8 章

多智能体：把独立工作拆开



多智能体让多个 AI 协作。本章采用主智能体分工与汇总的方式：主会话统一目标，子智能体完成限定工作。子任务可分别阅读大量材料，再返回结论与证据入口，减少主会话中的杂乱输出。优先并行独立调查；依赖紧密的修改统一推进。

从两个只读助手开始

1. **在主会话明确要求创建子智能体**。先确认工具支持，查看子任务运行记录。“只读”提示不是权限隔离：Codex 子任务沿用主会话的权限设置，不能靠角色说明单独限制；强制只读需相应设置主会话或独立运行环境。
2. **给各自不同任务**。一个查现有实现和可复用位置，另一个查相关方案、资料或验证风险。每个任务写清目标、材料位置、允许范围和返回内容，不假设它知道聊天历史或已读到项目规则。
3. **由主智能体统一判断**。依赖调查结论的修改，收齐依据后再做。并行写代码时约定文件范围、接口和完成条件，可用 Git worktree 分开工作目录；它不隔离权限或共享数据库。共享文件指定负责人，测试环境另行配置，合并后重查。

创建并行子任务

目标：[当前任务]，共同材料：[需求、调研、计划与任务位置]。先核对工具能力和权限；支持时创建两个只读子任务：A 查已有实现与复用位置，B 查 [方案、资料或验证风险]。材料不足先报缺口。
子任务不改文件、不扩展目标。若必须强制只读而无法配置，暂停启动并说明；角色提示不等于权限隔离。分别明确范围、输入、检查和返回格式：结论、位置、证据、未知项。主智能体在现有权限内推进独立工作；依赖结论的步骤等待结果，核对冲突后再推进。
若要并行修改，先明确授权、文件分工与合并检查。记录运行、完成或失败状态，未返回不算完成；不支持子智能体则返回可分别发送的任务说明，不模拟已启动。

分工让调查在各自上下文中进行，主会话保留决定与证据入口；它不能替代项目记录，也不保证消除信息损失。查看子任务是否实际启动，再收回来源、核对冲突。先从少量独立任务开始，比较总耗时、用量和返工后再扩量；智能体越多，重复工作与协调成本也可能越高。工具不支持时，可分会话调查后汇总。

第 9 章

让另一名智能体独立审查



审查意见也要核实，多个 AI 同意不能代替证据

重要功能完成、合并多份改动或正式使用前，可安排独立审查。审查者直接读取要求与当前文件，自行核对结果；只看实现者的完成总结，容易沿用同一个误解。

将实现与审查分开

1. **固定对象与材料**。记录版本、修改范围、需求和运行方法。优先用不继承实现对话的新会话或子智能体，只提供必要材料；复制原聊天历史的会话可能带入原有判断。
2. **按影响报告问题**。列出位置、触发条件、预期、实际和证据；优先处理正确性、安全与需求遗漏，风格偏好单列为可选。
3. **核对后修复，再复查**。实现者逐项验证意见，修复后交回当前版本。改动影响到的旧审查结论需重查；不为了回应每条建议而增加复杂度。

独立审查与复核

材料：[原始要求、当前需求、版本、修改范围、运行方法]。若你参与实现，且工具支持，交给未参与且不继承实现对话的审查者；否则返回交接说明，可先自查但不称独立审查。材料不全先列缺口。

不改被审查实现。先读文件；需运行且会写入时，使用获准测试副本或独立工作目录及测试数据。禁止写入则列未运行项，不绕过限制；目录分开不代表外部数据已隔离。

核对要求、改动与证据，重点查错误结果、数据或权限问题、被删除或放宽的检查及旧功能受损；长期项目再查重复与复杂度。逐项给位置、触发条件、影响和证据，区分已验证问题与风险。

输出给实现者的清单；获准且有工具时交接，否则由用户转交。修复后核对当前版本再复查，汇总已解决、仍存在及未验证项；范围内未发现实质问题可说明，不为凑数修改。

独立视角也需要证据

新审查者少受“自己刚做完”的判断影响，但仍可能犯相同错误。收到意见后，先核对触发步骤和证据，再交给实现者修复；修好后只复查受影响部分及项目要求的检查。多个 AI 同意不等于正确，拿不准的重要问题请有经验的人判断。

第 10 章

长任务怎样可靠地接着做



历史记录帮助恢复，当前结果仍需核对

长任务靠清晰规格、阶段验收和持续记录衔接。把想法整理成按依赖排序的规格队列，审核后逐项执行。中断、接近用量限制或反复偏离时保存进度，继续前核对状态。

结束前保存，重新开始后核对

`PROGRESS.md` 是文字进度文件，已有记录时沿用。结束前用第一张卡，重开后用第二张。

保存进度

请更新 [进度文件]：当前目标与决定、项目位置与版本、已改文件和未提交修改、检查及结果位置、失败和未验证项、未完成事项、恢复方法及下一步。无写权限则返回可保存文本，不称已落盘。
关键实验用索引链接假设、条件、操作、结果、放弃理由与证据。过期决定标明替代关系，不删除原始要求与证据，不把计划写成完成。列出各规格状态、依赖及下次核对条件；历史缺失标为未知，不凭记忆补造。

每完成一个可验收阶段就更新记录。恢复时重读当前规格、进度与相关实验；聊天中的“完成”不能代替可定位的结果。

读取进度并继续

读取 [进度文件] 及需求、方案和任务，核对目录、版本、已有改动与检查。无 Git 则用文件快照核对；记录缺失或不可读先报缺口，不猜进度，不覆盖他人工作。
从最先可执行的未完成事项继续，每段更新实际结果。仅沿用代码、软件、设置与数据均未受影响的结果；不确定则重查，项目规定的完整检查仍须执行。
按 [允许修改与运行范围] 和 [预算] 推进；受阻时继续独立可做部分。预算将尽先保存，不能写则返回交接文本。需要改变目标、扩大权限或解决重要未决选择时，暂停相关工作并说明。

用验收与停止条件约束迭代

让 AI 在预算内反复提出方案、执行、检查并保留有效结果，不能降低验收标准换取成功。达标则完成，预算将尽时保存；重复失败无新证据或遇到未决取舍时暂停相关工作。持续运行不保证收敛，恢复代码也不会撤回外部操作。

第 11 章

把有效做法写成规则和技能



长期规则保持简短，专项方法按需使用

遇到重复沟通或反复犯错，适合保存有效做法。安全边界从开始就设定；其他规则从实际问题中提炼。新增规则会增加读取量，不保证更准确，要用新的任务检验。

三类内容分别保存

保存方式	放什么，以及怎样使用
项目规则	特殊约定、允许范围、运行与检查入口；沿用 `AGENTS.md` 等工具实际识别的文件，不复制全库概览
Skill 技能	反复使用的方法、模板与材料；说明何时调用、输入和预期结果
自动程序	将稳定步骤保存为脚本；要在指定时机自动运行，再接入 Hook（钩子）机制

整理可复用规则与技能

根据 [认可结果、修改记录或重复任务] 提取复用要求；允许更新：[文件范围]。沿用规则，重复方法按当前工具规范整理为 Skill，固定步骤适合时写脚本。不支持 Skill 或不能写入时返回文档草稿，不声称已启用。

写明适用条件、输入输出、必要步骤、检查入口和失败处理；第三方技能或脚本先查来源与内容。保留已有有效约定，不新增无关安装。

报告路径与生效方法；需审核、信任或重启时说明。用获准的独立样例检验，无法调用则标未验证。自动检查验证触发及通过、失败、脚本出错时的行为，避免影响真实数据。

比较质量、沟通和返工；仅在允许范围合并重复或过时指导。必须执行的检查不能只靠提示词提醒。

保存标准，不固定所有思考步骤

从有效做法提炼规则。例如“允许本地提交，未经授权不推送远程仓库”可作项目约定；重复使用的调研方法可整理为 Skill。换一个同类任务检验效果，删除过时指导。

可为工具完成事件配置 Hook，调用脚本保存命令、结果和日志；后台任务另查真实退出状态。命令成功不等于验收通过。仅记录的 Hook 不拦截；需要阻止操作时，核对事前事件与失败处理并实际验证，事后记录不能撤销操作。

第 12 章

重复任务先小批试跑，再扩大



多个文件或任务需要相同处理时，先把单项流程做稳定，再扩大规模。批量处理是重复同一套规则；只有各项互不依赖、不会相互覆盖时，才进一步并行。

从小批结果确定整批规则

1. **先定清单和格式**。列出每项输入、预期输出、保存位置与完成条件，使用同一模板，保留原始材料。
2. **先处理少量代表项**。包含常见情况和容易失败的情况。检查内容与格式；AI 参与判断时，在测试副本上重复少量代表项，观察结论与质量是否稳定，再决定扩量。
3. **逐项记录并只补失败**。每项保留成功、失败、跳过或待确认状态及原因。规则、输入或运行环境改变后，需重查受影响项；同类错误反复出现时先停下修正流程。

小批试跑与批量执行

按 [规则和模板] 处理 [任务清单]，输出到 [位置]；允许操作与预算：[范围]。先试少量常见项和高风险项，展示结果与证据；AI 参与判断时重复代表项查稳定性。规则或完成条件不清先补齐，缺工具说明，不声称已处理。

为每项设唯一标识、输出格式、来源和完成条件。仅并行互不依赖项，避免覆盖原件和重复外部操作；重复试验只用副本或可安全重复的操作。记录状态、结果位置及失败原因；结果未知先核对，不盲目重试。续做未完成项，输入、规则或环境变化时重查受影响项。

逐项执行必要检查，再抽查内容；抽查不能替代逐项检查。试跑达标且在授权内再扩量，同类失败重复出现则暂停。汇总完整清单与缺口，比较总耗时、人工检查和返工，不只报告生成速度。

先修正流程，再扩大规模

先试少量，是为了在错误被重复很多次之前修正方法。看完试跑结果，再确认格式和判断标准；扩大后按清单核对成功项与缺口，失败项保留原因。规则改变时重查受影响项；不要只看到几份好结果，就认定整批都已完成。

十二章要点回顾

说清目标与取舍，用项目文件积累上下文，让每轮执行留下可验收的成果。

章	章节	核心思想与行动要点
01	把想法变成 可执行的任务	**先定义结果与取舍**。与 AI 讨论后整理规格，人审核本版做与不做，把偏好转成可观察的标准。
02	项目文件 怎样分工和衔接	**用文件积累上下文**。索引指向需求、依据与进度；压缩或换会话后按需重读，确认 AI 实际读到。
03	先读懂项目， 再决定怎么改	**先调研再选架构**。比较复用、改造与自建，查权威来源、设计理由与适用边界，验证关键未知。
04	用 Spec Kit 串起文件与执行	**让需求贯穿执行**。按当前步骤维护规格、计划与任务。关键取舍先核对，文档勾选不等于完成。
05	小步实现， 给出具体反馈	**每轮可检查、可恢复**。保存基线、接通真实操作；反馈写清位置、现状、目标和保持不变的部分。
06	从实际使用 判断是否完成	**预期来自原始要求**。走完整操作并核对保存记录；分清通过、失败和未验证，不改松检查换取通过。
07	出错时， 先定位再修改	**把失败变成资产**。记录假设、条件、操作和结果；保留无效路径与排除理由，修复后留下回归检查。
08	多智能体： 把独立工作拆开	**独立工作再并行**。分清输入、范围与证据，由主会话核对并汇总；先少量试用，再按收益与成本扩量。
09	让另一名智能体 独立审查	**审查意见也要验证**。审查者直接读要求与当前版本，按触发条件和证据报告问题；修复后再复核。
10	长任务怎样 可靠地接着做	**按规格队列持续推进**。写清依赖、验收和停止条件，每阶段保存结果与失败路径，接续前核对状态。
11	把有效做法 写成规则和技能	**只保留反复有用的做法**。规则写约定，Skill 存方法，脚本执行固定步骤；实际调用，检查是否减少返工。
12	重复任务 先小批试跑，再扩大	**先验证流程，再扩大规模**。试跑常见与困难项，逐项记录状态；续做未完成项，重查受影响项，衡量总耗时与返工。

贯穿十二章的主线：人负责目标、判断与最终验收；AI 在明确范围内执行；文件保存需求、决定和经验，证据说明完成到了哪里。先把项目组织清楚，再增加自动化。

方法与工具资料：DORA 小批量工作；Google SRE 排错；AWS 架构决策记录；Anthropic 上下文工程；OpenAI 子智能体；Spec Kit 官方流程。方法按场景选用，工具行为以版本和配置为准。